

MACHINE LEARNING PROJECT REPORT

BANKNOTE AUTHENTICATION USING
NAIVE BAYES, KNN, DECISION TREE ID3

Student: sheyda fathi

Instructor: Dr. Sarabadani

Course: Machine Learning

University: University of Science and Culture



ABSTRACT

This project implements three classical machine learning algorithms from scratch using only NumPy, pandas, and matplotlib. The algorithms are Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and Decision Tree ID3. The goal is to classify banknotes as authentic (0) or forged (1) using four features extracted from wavelet-transformed images. The dataset contains 1,372 samples with 4 numerical features (Variance, Skewness, Kurtosis, and Entropy). Models are evaluated using 5-fold cross-validation, paired t-tests, and learning curves. Results show that KNN achieves the best performance with 99.85% accuracy.



CHAPTER 1: METHODOLOGY

1.1 Problem Formulation

The banknote authentication problem is formulated as a supervised concept learning task:

Instance Space (X): Each instance has 4 numerical features: Variance, Skewness, Kurtosis, and Entropy. Thus $X = \mathbb{R}^4$

Label Space (Y): Binary labels where 0 = authentic banknote and 1 = forged banknote. Thus $Y = \{0, 1\}$

Target Function (f): $f: X \rightarrow Y$ mapping features to authenticity status

Training Data: 1,372 labeled examples ($x, f(x)$) split into 70% training, 15% validation, and 15% test

Evaluation Criteria: Accuracy, Precision, Recall, F1 Score with 5-fold cross-validation

1.2 Data Preprocessing

Min-Max Normalization

For KNN algorithm (distance-based), feature normalization is required:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Parameters x_{min} and x_{max} are computed only on training data to prevent data leakage.

Data Split:

Data Split		
Split	Percentage	Number of Samples
Training	70%	960 samples
Validation	15%	206 samples
Test	15%	206 samples

1.3 Algorithm 1: Gaussian Naive Bayes

This algorithm is based on Bayes' theorem with the naive assumption of feature independence:

$$P(C_k|x) \propto P(C_k) \prod_{i=1}^n P(x_i|C_k)$$

For continuous data, Gaussian probability density function is used:

$$P(x_i|C_k) = \frac{1}{\sqrt{2\pi\sigma_{ik}^2}} \exp\left(-\frac{(x_i - \mu_{ik})^2}{2\sigma_{ik}^2}\right)$$

Implementation Steps:

1. Calculate mean and variance for each feature per class
2. Calculate prior probability for each class
3. Use log-probabilities to prevent numerical underflow
4. Select class with highest log-posterior probability

1.4 Algorithm 2: K-Nearest Neighbors (KNN)

KNN is an instance-based lazy learner that stores all training data:

$$d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$$

Implementation Steps:

1. Store all training data (lazy learner)
2. Calculate Euclidean distance to all training samples
3. Find k samples with smallest distance
4. Simple majority voting (no weights) for final label
5. Test k = 1, 3, 5, 7 and select best k using validation set

1.5 Algorithm 3: Decision Tree ID3

Decision tree using Information Gain criterion with pre-pruning (max_depth=5):

Entropy:

$$H(S) = - \sum_{c=1}^C p_c \log_2(p_c)$$

Information Gain:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Implementation Steps:

1. Discretize continuous features using mean threshold
2. Calculate entropy of the dataset
3. Calculate Information Gain for each feature
4. Select feature with highest Information Gain for splitting
5. Stop when all samples same class or max_depth=5 reached

1.6 Implementation Challenges and Solutions

Challenge	Solution
Numerical underflow in Naive Bayes	Use log-probabilities instead of raw probabilities
Different feature scales in KNN	Min-Max normalization on all features
Continuous features in Decision Tree	Discretization using mean threshold (low/high)
Data leakage in normalization	Compute parameters only on training data



CHAPTER 2: NUMERICAL RESULTS

2.1 5-Fold Cross-Validation Results

Algorithm	Accuracy	Precision	Recall	F1 Score
Naive Bayes	0.8396 ± 0.0250	0.8452 ± 0.0381	0.7825 ± 0.0225	0.8125 ± 0.0252
KNN	0.9985 ± 0.0018	0.9971 ± 0.0037	1.0000 ± 0.0000	0.9985 ± 0.0019
Decision Tree	0.9453 ± 0.0189	0.9512 ± 0.0224	0.9389 ± 0.0182	0.9450 ± 0.0195

2.2 Paired t-test Results

Comparison	t-statistic	p-value	Significant (p < 0.05)
Naive Bayes vs KNN	-12.45	0.0002	✓ Yes
Naive Bayes vs Decision Tree	-8.92	0.0008	✓ Yes
KNN vs Decision Tree	5.23	0.0065	✓ Yes

2.3 best algorithm selection

1. Best Algorithm: KNN
2. Mean Accuracy: 0.9985 ± 0.0018
3. Lowest standard deviation among all algorithms
4. Perfect Recall score (1.0000)
5. Statistically significant difference from both other algorithms



CHAPTER 3: PLOTS AND FIGURES

3.1 Algorithm Comparison Bar Chart with Error Bars

[Insert image: algorithm_comparison.png]

3.2 Learning Curves Comparison

[Insert image: learning_curves_comparison.png]

Learning Curve - Naive Bayes

[Insert image: learning_curve_naive_bayes.png]

Learning Curve - KNN

[Insert image: learning_curve_knn.png]

Learning Curve - Decision Tree

[Insert image: learning_curve_decision_tree.png]

CHAPTER 4: ANALYSIS AND CONCLUSION

4.1 Learning Curve Interpretation

Algorithm	Accuracy at 20%	Accuracy at 100%	Improvement	Interpretation
Naive Bayes	0.7523	0.8396	8.7%	High Bias (Underfitting)
KNN	0.9856	0.9985	1.3%	Low Variance (Very Stable)
Decision Tree	0.8521	0.9453	9.3%	Moderate Variance

Analysis:

Naive Bayes: Underfitting due to feature independence assumption. Limited improvement with more data.

KNN: Excellent and stable performance from the beginning. Very low variance.

Decision Tree: Significant improvement with more data. Pre-pruning (max_depth=5) prevents overfitting.

4.2 Paired t-test Analysis

All p-values are less than 0.05 significance level, therefore:

Null hypothesis (H_0) of equal algorithm performance is rejected

KNN performs significantly better than both other algorithms

Decision Tree performs significantly better than Naive Bayes

4.3 Limitations of Each Algorithm on This Dataset

Naive Bayes

Assumes feature independence - banknote features may be correlated

Gaussian distribution assumption may not perfectly fit the data

Lower accuracy (84%) compared to other algorithms

KNN

Lazy learner - stores all training data (memory intensive)

Sensitive to irrelevant features

Requires optimal k selection

High prediction time for large datasets

Decision Tree

Pre-pruning (depth=5) limits model complexity

Discretization with mean threshold loses information

Higher variance than KNN on this dataset

4.4 Suggestions for Improving the Weakest Algorithm (Naive Bayes)

Naive Bayes is the weakest algorithm with 84% accuracy. Improvement suggestions:

1. Use Kernel Density Estimation (KDE) instead of Gaussian distribution

2. Apply Feature Selection to remove redundant features

3. Use Box-Cox Transformation to make feature distributions more Gaussian

4. Consider Complement Naive Bayes for imbalanced classes

5. Use Ensemble Methods like Averaged One-Dependence Estimators (AODE)

4.5 Final Conclusion

Three machine learning algorithms were implemented from scratch and evaluated on the banknote authentication dataset.

Results show:

Rank	Algorithm	Accuracy	Recommendation
1	KNN	99.85%	Recommended for practical use
2	Decision Tree	94.53%	Acceptable performance
3	Naive Bayes	83.96%	Needs improvement

All performance are statistically significant ($p < 0.05$).

KNN is the best algorithm for this banknote authentication task.

Appendix: Project Files

project.py - Complete Python code

README.txt - Execution instructions

output/ folder - 6 plot images